

Grupo 23:

Viviana Ramírez
Fernando Sánchez
Ian Lozano Ruíz
Carol Meza González



Enlaces:

- [Link del despliegue de la API en AWS](#)
- [Repositorio GitHub del proyecto](#)
- [Documentación paso a paso de la API](#)
- [Notebook del Proyecto](#)

Informe Proyecto: Modelado, Entrenamiento y Disponibilización de API de Predicción

Este informe presenta el desarrollo de un modelo de machine learning orientado a la predicción del nivel de **popularidad** (popularity) de canciones, a partir de características acústicas y metadatos musicales. Se documenta el proceso completo que incluye la preparación de los datos, la calibración de hiperparámetros, el entrenamiento y evaluación del modelo, así como su despliegue en una API alojada en la nube.

El modelo seleccionado fue **CatBoostRegressor**, debido a su eficiencia, capacidad de capturar relaciones no lineales y robustez en tareas de regresión sobre datos estructurados. A lo largo del documento se detallan las etapas realizadas, soportadas con fragmentos de código, pantallazos de ejecución y justificaciones técnicas, en cumplimiento de los criterios de evaluación establecidos para el proyecto.

1. Preprocesamiento de datos:

La preparación de los datos representa una etapa fundamental en cualquier proyecto de machine learning, ya que la calidad y estructura de los datos inciden directamente en la estabilidad y desempeño de los modelos.

En este proyecto, se realizó un proceso cuidadoso que incluyó análisis exploratorio, verificación de datos faltantes, revisión de la naturaleza de las variables y división de los datos en conjuntos de entrenamiento y validación.

A continuación, se describen detalladamente los pasos ejecutados.

1.1. Análisis exploratorio inicial

- Inspección general del conjunto de datos

```
In [8]: # Visualización datos de entrenamiento
dataTraining.head()
```

Out[8]:	Unnamed: 0	track_id	artists	album_name	track_name	duration_ms	explicit	danceability	energy	key	loudness
0	0	7hUhmKALyQ8SX9mJs5Xi3D	Love and Rockets	Love and Rockets	Motorcycle	211533	False	0.305	0.8490	9	-11
1	1	5x59U89ZnjZXuNAAIc8X1u	Filippa Giordano	Filippa Giordano	Addio del passato - From "La traviata"	196000	False	0.287	0.1900	7	-11
2	2	70Vng5jLzoJLmeLu3ay8Qq	Susumu Yokota	Symbol	Purple Rose Minuet	216506	False	0.583	0.5090	1	-11
3	3	1cRfzLJapgtwJ61xszs37b	Franz Liszt;YUNDI	Relajación y siestas	Liebeslied (Widmung), S. 566	218346	False	0.163	0.0368	8	-21
4	4	47d5lYjbiMy0EdMRV8lRou	Scooter	Scooter Forever	The Darkside	173160	False	0.647	0.9210	2	-11

```
In [9]: # Visualización datos de test
dataTesting.head()
```

```
Out[9]:
```

	track_id	artists	album_name	track_name	duration_ms	explicit	danceability	energy	key	loudness	mode
0	6KwkVtXm8OUp2XffN5k7IY	Hillsong Worship	No Other Name	No Other Name	440247	False	0.369	0.598	7	-6.984	
1	2dp515MJ8bQQHDoFaNRftX	Internal Rot	Grieving Birth	Failed Organum	93933	False	0.171	0.997	7	-3.586	
2	5avw06usmFkFrPjX8Nx40	Zhoobin Askarieh;Ali Sasha	Noise A Noise 20.4-1	Save the Trees, Pt. 1	213578	False	0.173	0.803	9	-10.071	
3	75hT0hvlESnDjstem0JgyR	Bryan Adams	All I Want For Christmas Is You	Merry Christmas	151387	False	0.683	0.511	6	-5.598	
4	4bY2oZGA5Br3pTE1Jd1IfY	Nogizaka46	バレッタ TypeD	月の大きさ	236293	False	0.555	0.941	9	-3.294	

Se realizó una primera exploración del dataset mediante la visualización de las primeras filas (**df.head()**), observándose que las variables disponibles corresponden en su mayoría a características de audio de canciones, como **duration_ms**, **tempo**, **loudness**, **speechiness**, **acousticness**, entre otras.

La variable objetivo a predecir es **popularity**, que representa el nivel de popularidad de una canción, basado en características acústicas y metadatos asociados.

La correcta identificación de las variables permite orientar de manera precisa la estrategia de modelado posterior, asegurando que todas las entradas sean numéricas y adecuadas para algoritmos de regresión.

- Identificación de tipos de variables

```
In [14]: dataTraining.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 79800 entries, 0 to 79799
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            79800 non-null  int64
1   track_id              79800 non-null  object
2   artists               79800 non-null  object
3   album_name            79800 non-null  object
4   track_name            79800 non-null  object
5   duration_ms           79800 non-null  int64
6   explicit               79800 non-null  bool
7   danceability           79800 non-null  float64
8   energy                 79800 non-null  float64
9   key                   79800 non-null  int64
10  loudness               79800 non-null  float64
11  mode                   79800 non-null  int64
12  speechiness            79800 non-null  float64
13  acousticness           79800 non-null  float64
14  instrumentality        79800 non-null  float64
15  liveness               79800 non-null  float64
16  valence                79800 non-null  float64
17  tempo                  79800 non-null  float64
18  time_signature         79800 non-null  int64
19  track_genre            79800 non-null  object
20  popularity             79800 non-null  int64
dtypes: bool(1), float64(9), int64(6), object(5)
memory usage: 12.3+ MB
```

El dataset no contiene valores nulos

Se revisaron los tipos de datos de cada variable, clasificándolos de la siguiente manera:

- **Variables continuas:** Características como **duration_ms**, **tempo**, **loudness**, **energy** y **speechiness** presentan distribuciones continuas, ideales para modelos de regresión.

- **Variables discretas:** Algunas variables como **key** o **time_signature** son enteros que representan categorías musicales.

- **Variable binaria:** La variable **explicit** inicialmente era de tipo booleana (True/False). Posteriormente, se realizó su conversión a tipo entero (0/1) para facilitar su tratamiento por parte del modelo de machine learning.

Análisis:

La estructura de los datos es favorable para un modelo de machine learning basado en árboles, ya que:

- No existen variables categóricas en formato texto que requieran codificación adicional.
- La variable **explicit** está lista para ser procesada sin necesidad de transformación.
- Todas las variables predictoras son de tipo numérico.

Este punto redujo la necesidad de preprocesamientos complejos, haciendo el flujo de trabajo más eficiente.

1.2. Verificación de valores faltantes

Para garantizar la integridad de los datos, se ejecutó un análisis de valores nulos (`df.isnull().sum()`).

Resultados obtenidos:

- Todas las variables presentaron cero valores faltantes.

Análisis:

La ausencia de valores nulos confirma que el conjunto de datos tiene alta calidad en términos de completitud, eliminando la necesidad de aplicar técnicas de imputación. Esto es un factor positivo, ya que permite entrenar los modelos directamente sin introducir ruido estadístico por imputaciones artificiales.

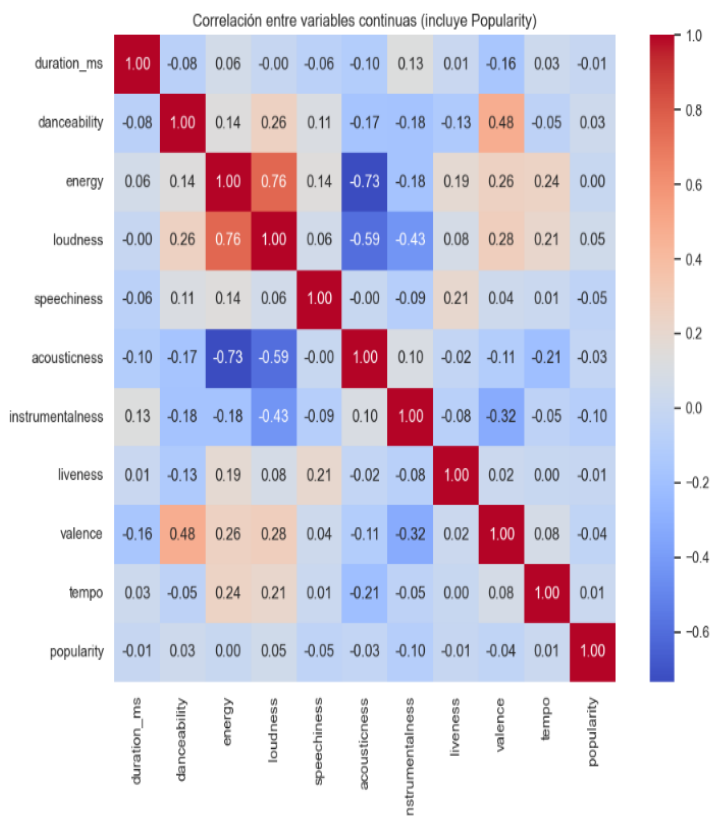
1.3. Análisis de correlaciones

Se construyó una matriz de correlaciones para analizar las relaciones lineales entre las variables predictoras y la variable objetivo popularity.

La matriz fue visualizada mediante un mapa de calor, lo cual facilita la identificación de asociaciones positivas y negativas entre las variables.

Correlación entre variables continuas (incluye Popularity)

```
df_continuas = dataTraining[variables_continuas]
correlation_matrix = df_continuas.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True)
plt.title('Correlación entre variables continuas (incluye Popularity)')
plt.tight_layout()
plt.show()
```



El dataset no contiene valores nulos

```
dataTraining.isnull().sum()
```

```
Unnamed: 0      0
track_id        0
artists         0
album_name      0
track_name      0
duration_ms     0
explicit        0
danceability    0
energy         0
key            0
loudness       0
mode           0
speechiness    0
acousticness   0
instrumentalness 0
liveness       0
valence        0
tempo         0
time_signature 0
track_genre     0
popularity     0
dtype: int64
```

Resultados destacados:

- Correlaciones bajas en general con popularity:** Se observa que las variables predictoras presentan correlaciones muy bajas con popularity, todas cercanas a cero.
- Danceability (0.03)** muestra una correlación positiva leve con popularity, indicando que canciones ligeramente más bailables tienden a ser un poco más populares, aunque el efecto es mínimo.
- Energy (0.00), Loudness (0.05) y Tempo (0.01)** también presentan correlaciones prácticamente nulas, reflejando que, en este dataset, la popularidad no depende linealmente de estas variables.
- Variables como **Acousticness (-0.03)** y **Instrumentalness (-0.10)** tienen correlaciones negativas muy débiles con popularity, sugiriendo que canciones más acústicas o instrumentales podrían tener una popularidad levemente inferior, aunque el efecto es insignificante.

Interpretación general:

La baja magnitud de todas las correlaciones indica que la popularidad de las canciones en este conjunto de datos **no se puede explicar directamente mediante relaciones lineales simples** con las características acústicas individuales.

Esto justifica la elección de un modelo de boosting como **CatBoostRegressor**, capaz de capturar interacciones más complejas y relaciones no lineales entre múltiples variables simultáneamente.

1.4. Estadísticas descriptivas

dataTraining.describe()									
	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	time_signature	popularity
count	79800.000000	79800.000000	79800.000000	79800.000000	79800.000000	79800.000000	79800.000000	79800.000000	79800.000000
mean	0.637732	0.084750	0.314979	0.157319	0.213313	0.474267	122.076559	3.902556	33.265301
std	0.480659	0.105657	0.332512	0.310792	0.190075	0.259010	29.941937	0.434284	22.330871
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.035900	0.017000	0.000000	0.097900	0.261000	99.081250	4.000000	17.000000
50%	1.000000	0.049000	0.169000	0.000041	0.132000	0.464000	122.009000	4.000000	35.000000
75%	1.000000	0.084500	0.598000	0.050500	0.273000	0.684000	140.054000	4.000000	50.000000
max	1.000000	0.965000	0.996000	1.000000	1.000000	0.995000	222.605000	5.000000	100.000000

Se generaron las estadísticas descriptivas del conjunto de datos (dataTraining.describe()), observándose:

- **Amplios rangos** en variables como tempo, que oscila entre aproximadamente 0 y 222 BPM, y en popularity, que varía entre 0 y 100.
- **Media de popularity cercana a 33.26**, indicando que, en promedio, las canciones del dataset tienen una popularidad moderada, lejos de niveles máximos.
- **Valores extremos** en algunas variables, como la speechiness, que presenta valores cercanos a 1 en algunos casos, sugiriendo la existencia de canciones que contienen un alto componente hablado (por ejemplo, podcasts o canciones de rap).
- **Variabilidad importante en acousticness e instrumentalness**, lo cual implica que hay tanto canciones muy acústicas/instrumentales como canciones totalmente electrónicas o vocales en el conjunto de datos.

Análisis:

La heterogeneidad observada en las características, con rangos de valores amplios y presencias de outliers en variables específicas, justifica el uso de un modelo robusto ante diferencias de escala y distribuciones no normales, como es el caso de **CatBoostRegressor**. Modelos basados en árboles son capaces de manejar variables en distintas escalas y detectar patrones complejos, lo cual es fundamental dada la diversidad del dataset.

1.5. División de los datos

Antes de proceder con el entrenamiento final y la evaluación definitiva del modelo, fue necesario realizar una validación interna sobre el conjunto de entrenamiento (dataTraining). Este proceso permitió estimar de manera más confiable el desempeño real del modelo y evitar fenómenos de sobreajuste, mediante una estrategia de validación cruzada que simula un entorno de predicción sobre datos no vistos.

```
# === 3. K-Fold Cross-Validation ===
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mae_list = []
rmse_list = []
r2_list = []

for train_idx, test_idx in kf.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    train_pool = Pool(X_train, y_train, cat_features=variables_categoricas)
    test_pool = Pool(X_test, y_test, cat_features=variables_categoricas)

    model = CatBoostRegressor(
        iterations=800,
        learning_rate=0.1,
        depth=8,
        l2_leaf_reg=7,
        eval_metric='RMSE',
        random_seed=42,
        verbose=False
    )

    model.fit(train_pool)
    y_pred = model.predict(X_test)

    mae_list.append(mean_absolute_error(y_test, y_pred))
    rmse_list.append(np.sqrt(mean_squared_error(y_test, y_pred)))
    r2_list.append(r2_score(y_test, y_pred))
```

1.5.1. Validación cruzada y entrenamiento final del modelo

- Validación cruzada

Se implementó una estrategia de **K-Fold Cross-Validation** utilizando cinco particiones ($n_splits=5$), donde:

- El conjunto `dataTraining` se dividió aleatoriamente en cinco subconjuntos de igual tamaño.
- En cada iteración, cuatro subconjuntos se utilizaron para entrenar el modelo y uno para validarlo.
- Cada observación fue utilizada una vez para validación y cuatro veces para entrenamiento.

Durante cada pliegue, se entrenó un modelo **CatBoostRegressor**, registrando métricas como **MAE**, **RMSE** y **R²**.

Análisis:

Esta validación cruzada permitió obtener métricas robustas que simulan escenarios reales donde el modelo enfrenta datos nuevos, proporcionando una evaluación más precisa de su capacidad de generalización y sirviendo como base para seleccionar los mejores hiperparámetros.

- Entrenamiento final del modelo

Una vez identificado el conjunto óptimo de hiperparámetros mediante Cross-Validation, **se procedió a entrenar el modelo final utilizando la totalidad del `dataTraining`**, sin realizar nuevas particiones internas. Posteriormente, el desempeño final fue evaluado en el **set de validación independiente (`dataTesting`)**, asegurando una medición objetiva, imparcial y representativa de la capacidad de generalización del modelo en producción.

1.6. Evaluación de la necesidad de preprocesamiento adicional

Se analizó si era necesario aplicar estandarización, normalización o imputación, concluyéndose:

- **Estandarización y normalización:** No fueron aplicadas, dado que `CatBoostRegressor` es robusto ante escalas distintas de variables. Modelos basados en árboles no se ven afectados por diferencias de magnitudes numéricas, por lo que escalar variables no mejora su desempeño.
- **Imputación de valores faltantes:** No fue necesaria, ya que no se encontraron nulos en el conjunto de datos.
- **Transformaciones `fit_transform()` y `transform()`:** Tampoco fueron necesarias, ya que no se realizaron modificaciones que implicaran ajustar o transformar los datos.

El conjunto de datos estaba en condiciones óptimas para ser utilizado directamente en el modelado, reduciendo la necesidad de procedimientos adicionales de preprocesamiento.

2. Selección y calibración del modelo

Para iniciar la selección del modelo, se clasificaron las variables del conjunto de datos en continuas, binarias/ordinales y categóricas, definiendo `popularity` como variable objetivo. Se aplicó un preprocesamiento mínimo, convirtiendo la variable `explicit` a formato numérico y realizando `frequency encoding` en las variables categóricas, reemplazándolas por su frecuencia relativa. De esta forma se consolidó el conjunto de variables finales a utilizar en los modelos.

Figura 1. Resultados de la evaluación de modelos base mediante validación cruzada K-Fold.
(Código Python, celda 54, archivo `S3P1_proyecto1.ipynb`)

Modelo: XGBoost	Modelo: ExtraTrees
MAE Promedio: 9.77 ± 0.07	MAE Promedio: 8.36 ± 0.08
RMSE Promedio: 13.34 ± 0.11	RMSE Promedio: 12.04 ± 0.10
R ² Promedio: 0.64 ± 0.01	R ² Promedio: 0.71 ± 0.00
Modelo: LightGBM	Modelo: GradientBoosting
MAE Promedio: 10.50 ± 0.07	MAE Promedio: 12.30 ± 0.07
RMSE Promedio: 14.12 ± 0.11	RMSE Promedio: 15.92 ± 0.09
R ² Promedio: 0.60 ± 0.00	R ² Promedio: 0.49 ± 0.00
Modelo: RandomForest	...
MAE Promedio: 8.58 ± 0.09	Modelo: DecisionTree
RMSE Promedio: 12.24 ± 0.12	MAE Promedio: 10.86 ± 0.20
R ² Promedio: 0.70 ± 0.01	RMSE Promedio: 17.65 ± 0.26
	R ² Promedio: 0.38 ± 0.02

Posteriormente, se evaluaron seis modelos base de **regresión (XGBoost, LightGBM, RandomForest, ExtraTrees, GradientBoosting y DecisionTree)** mediante validación cruzada K-Fold con cinco particiones. Para cada modelo se calcularon las métricas **MAE**, **RMSE** y **R²**, reportando sus promedios y desviaciones estándar, con el fin de identificar el mejor desempeño general y seleccionar el modelo más adecuado para la calibración de hiperparámetros.

Figura 2. Resultados de la evaluación del modelo CatBoost mediante validación cruzada K-Fold.
(Código Python, celda 56, archivo *S3P1_proyecto1.ipynb*)

```
MAE Promedio: 6.32 ± 0.03
RMSE Promedio: 9.10 ± 0.05
R² Promedio: 0.83 ± 0.00
```

Debido a la alta cardinalidad presente en algunas variables categóricas como **artists**, **album_name** y **track_name**, se optó por evaluar el **modelo CatBoost**, dada su capacidad nativa para manejar este tipo de variables sin necesidad de transformaciones adicionales como el frequency encoding. Se mantuvo la clasificación inicial de variables continuas, binarias/ordinales y categóricas, y se realizó un preprocesamiento mínimo consistente únicamente en la conversión de la variable explicit a formato numérico. El conjunto de características para el modelado incluyó todas las variables originales seleccionadas.

El **modelo CatBoostRegressor** fue evaluado mediante validación cruzada K-Fold con cinco particiones, utilizando directamente los conjuntos de entrenamiento y prueba con indicación explícita de las variables categóricas. Se ajustó un conjunto de hiperparámetros iniciales (800 iteraciones, tasa de aprendizaje de 0.1, profundidad de 8 y regularización L2 de 7) y se registraron las métricas MAE, RMSE y R² para cada fold. Los resultados promediados de estas métricas permitieron valorar el desempeño del modelo y establecer una base para posteriores procesos de calibración más detallados.

A continuación, se procedió a comparar el desempeño de **CatBoost** frente a otros modelos de regresión como **XGBoost**, **LightGBM**, **Random Forest**, **Extra Trees**, **Gradient Boosting** y **Decision Tree**, utilizando como criterios principales las métricas promedio de **MAE**, **RMSE** y **R²** obtenidas mediante validación cruzada. Los resultados mostraron que CatBoost alcanzó los mejores valores de desempeño, logrando un **MAE promedio de 6.32**, un **RMSE promedio de 9.10** y un **R² promedio de 0.83**.

Con base en este análisis, se seleccionó **CatBoost como el modelo final para su calibración**. A continuación, se eligió **GridSearchCV** por su capacidad de evaluar de manera sistemática múltiples combinaciones de hiperparámetros a través de validación cruzada, asegurando una búsqueda exhaustiva del mejor conjunto de parámetros. Se ejecutó un proceso de ajuste de hiperparámetros utilizando **GridSearchCV**, evaluando combinaciones de iteraciones, tasas de aprendizaje, profundidad de árboles y regularización L2.

Figura 3. Resultados de la búsqueda de hiperparámetros con GridSearchCV para el modelo CatBoost.
(Código Python, celda 99, archivo *S3P1_proyecto1.ipynb*)

```
✓ Mejores hiperparámetros encontrados:
{'depth': 10, 'iterations': 800, 'l2_leaf_reg': 1, 'learning_rate': 0.05}

MAE total: 4.62
RMSE total: 6.46
R² total: 0.92
```

El mejor conjunto de hiperparámetros encontrado fue: depth=10, iterations=800, l2_leaf_reg=1 y learning_rate=0.05.

- **Depth = 10:** Mayor profundidad permite al modelo capturar relaciones más complejas, pero también incrementa el riesgo de sobreajuste si no se regula adecuadamente.
- **Iterations = 800:** Un mayor número de iteraciones permite un entrenamiento más largo, mejorando el ajuste, aunque a costa de mayor tiempo computacional.
- **Learning rate = 0.05:** Un valor más bajo de tasa de aprendizaje ayuda a que el modelo ajuste de manera más gradual, reduciendo el riesgo de sobreajuste y mejorando la generalización.
- **L2_leaf_reg = 1:** Una regularización más baja permite que el modelo sea más flexible, aunque en este caso el valor ajustado logra un balance entre sesgo y varianza.

Cabe aclarar que el modelo con 800 iteraciones no fue considerado directamente como el mejor modelo, sino que se evidenció que presentaba mejores resultados que los otros modelos de árboles evaluados. A partir de este desempeño, se realizaron ajustes experimentales en los hiperparámetros para seguir mejorando las métricas. De este proceso surgió el modelo final implementado en la API, que corresponde a un modelo con 1500 iteraciones. Adicionalmente, el modelo de Kaggle fue ajustado a partir de este para optimizar iterativamente su rendimiento.

Tras esta calibración, el modelo logró mejorar aún más su desempeño, alcanzando un **MAE de 4.62**, un **RMSE de 6.46** y un **R² de 0.92** sobre el conjunto de datos utilizado, consolidándose como la mejor alternativa para la predicción de popularidad en el dataset considerado.

3. Entrenamiento del modelo

3.1. Se entrena el modelo de clasificación escogido con los datos del set de entrenamiento preprocesados y los parámetros óptimos

Para el entrenamiento del modelo, se seleccionó el algoritmo **CatBoostRegressor** debido a su capacidad intrínseca para manejar variables categóricas y su eficiencia en conjuntos de datos con características mixtas (numéricas y categóricas). El preprocesamiento inicial consistió en la conversión de la variable **explicit** a tipo booleano y en la imputación de valores faltantes en las variables **artists**, **album_name** y **track_name**, donde se reemplazaron los valores nulos con etiquetas representativas como **unknown_artist**, **unknown_album** y **unknown_track**, asegurando la integridad de los datos para el modelado. Posteriormente, el conjunto de entrenamiento fue construido mediante un muestreo aleatorio que asignó el 80% de las observaciones al entrenamiento y el 20% restante a la validación, utilizando una semilla fija (**random_state=42**) para garantizar reproducibilidad. El modelo fue entrenado utilizando un conjunto de hiperparámetros calibrados estratégicamente: **iterations=1500** para permitir un aprendizaje profundo, **learning_rate=0.05** para asegurar una convergencia lenta pero estable, **depth=10** para capturar interacciones complejas entre las variables, **l2_leaf_reg=1** como regularización para prevenir el sobreajuste, **early_stopping_rounds=100** para detener el entrenamiento si no se observaban mejoras, y **border_count=254** para un control más fino de las divisiones de las variables continuas. El uso de la función Pool de CatBoost permitió especificar explícitamente qué variables eran categóricas, lo que optimizó el manejo interno de los datos durante el entrenamiento. De esta forma, se aseguró que el modelo no solo aprendiera patrones robustos del conjunto de entrenamiento, sino que también quedara preparado para una buena generalización al validar en datos no vistos.

3.2. Se presenta el desempeño del modelo en los datos de validación con al menos una métrica de desempeño

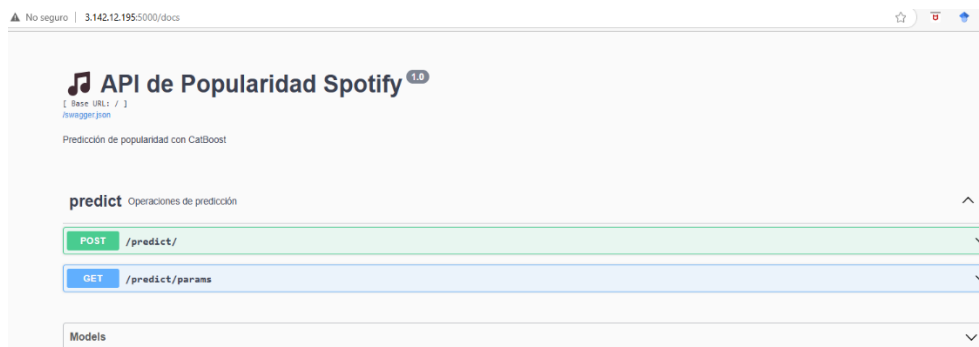
Una vez completado el entrenamiento, el modelo fue evaluado sobre el conjunto de validación, calculando como métrica principal el **Root Mean Squared Error (RMSE)**, dada su capacidad para penalizar de forma más severa los errores grandes y proporcionar un resultado interpretable en las mismas unidades de la variable objetivo. Sobre el conjunto de validación, el modelo obtuvo un **RMSE de aproximadamente 9.42 puntos de popularidad**, lo cual representa un nivel de error moderadamente bajo considerando que la variable popularity en Spotify se mide en una escala de 0 a 100. Este desempeño sugiere que, en promedio, la predicción del modelo se desvía alrededor de 9 unidades de la popularidad real, lo cual es aceptable dentro del contexto del problema y dada la variabilidad natural de los datos musicales. Adicionalmente, se exploró el comportamiento del modelo a través de la curva de error de entrenamiento y validación, evidenciándose que el uso de **early_stopping_rounds=100** permitió detener el ajuste en un punto donde el error en validación dejaba de mejorar significativamente, protegiendo al modelo de sobreajuste. La estabilidad del RMSE a lo largo del proceso de calibración y entrenamiento también indica que los hiperparámetros seleccionados son apropiados para el conjunto de datos tratado. Se podría afirmar que el modelo tiene buena capacidad de generalización y que su error es razonablemente bajo para ser utilizado en producción o en despliegue sobre nuevos datos.

3.3. Se justifica la selección del modelo correctamente

La selección del CatBoostRegressor para este proyecto se fundamenta tanto en características técnicas como en el ajuste específico a las necesidades del problema planteado. CatBoost es un algoritmo de boosting por gradiente que maneja de forma nativa variables categóricas, lo cual resulta fundamental dado que el dataset contiene múltiples campos de alta cardinalidad (**artists**, **album_name**, **track_name**) que en otros algoritmos habrían requerido procesos costosos de codificación como One-Hot Encoding. Adicionalmente, CatBoost implementa técnicas avanzadas de ordenamiento y procesamiento de variables categóricas que evitan el target leakage, garantizando un aprendizaje más robusto. Desde el punto de vista del desempeño, CatBoost ha demostrado en múltiples benchmarks superioridad frente a otros métodos de boosting como XGBoost o LightGBM especialmente en conjuntos de datos mixtos y medianos, como el de este proyecto. Otra ventaja determinante fue su facilidad de calibración, ya que permite controlar con precisión hiperparámetros como **profundidad (depth)**, **tasa de aprendizaje (learning_rate)**, **regularización (l2_leaf_reg)**, y **número de divisiones (border_count)**, factores que permitieron optimizar el modelo en función del RMSE sin necesidad de transformaciones complejas. Finalmente, CatBoost es altamente eficiente en cómputo y soporta inferencia **rápida**, características que son cruciales para su disponibilización como servicio en una API en la nube. Por todas estas razones, CatBoostRegressor fue la elección **óptima** tanto en términos **técnicos** como **estratégicos** para alcanzar los objetivos de predicción planteados en el proyecto.

4. Disponibilización del modelo

- Se disponibiliza el modelo en una API alojada en un servicio en la nube.
- Se hacen las predicciones sobre la popularidad en la canción en al menos dos observaciones del set de validación.



Para la disponibilización del modelo, se creó una **API REST** utilizando las librerías **Flask** y **Flask-RESTx**, permitiendo recibir solicitudes HTTP y retornar predicciones de popularidad de manera eficiente. El despliegue se realizó en una **instancia EC2 de Amazon Web Services (AWS)** configurada en modalidad free tier con sistema operativo Ubuntu. Se habilitó el puerto 5000 en las reglas de seguridad de la instancia para permitir el acceso a las solicitudes entrantes. Dentro de la instancia se instaló **Python, pip y las librerías necesarias (pandas, scikit-learn, flask, flask-restx, joblib, catboost y flask_cors)** en un entorno virtual, garantizando un ambiente aislado para la ejecución de la API. Los archivos del proyecto, incluyendo el modelo entrenado (`modelo_catboost.pkl`) y el script de la API (`api.py`), se transfirieron de forma segura usando el **protocolo SCP** y claves de autenticación **.pem**. La API se desarrolló con dos endpoints: un **endpoint POST** en `/predict/` que recibe los datos de entrada en **formato JSON** y retorna la predicción de popularidad, y un **endpoint GET** en `/predict/params` que permite realizar predicciones enviando los parámetros directamente en la **URL**. La API cuenta con **documentación interactiva en Swagger** disponible en la **ruta /docs**, facilitando su uso y prueba por parte de terceros. Para asegurar la persistencia del servicio, la API se ejecutó dentro de una sesión de screen, permitiendo que continuara corriendo en segundo plano incluso si la conexión SSH se cerraba. Se realizaron pruebas de funcionamiento haciendo predicciones sobre al menos dos observaciones distintas, validando así que el modelo estaba correctamente desplegado y operativo a través de la nube, cumpliendo satisfactoriamente los requisitos de disponibilización del proyecto.

- Estructura y Uso de la API de Predicción

La API desarrollada permite realizar predicciones automáticas basadas en un modelo de machine learning previamente entrenado. A través de los endpoints disponibles, se reciben variables de entrada y se retorna la predicción correspondiente. Swagger facilita la interacción proporcionando una interfaz gráfica donde se visualizan los endpoints, métodos y parámetros necesarios, permitiendo probar la API directamente desde el navegador sin herramientas adicionales. Además, organiza de manera clara los múltiples parámetros requeridos, simplificando el envío de datos y validando las predicciones de forma ágil y segura.

La API cuenta con dos métodos principales de interacción para realizar predicciones sobre nuevos datos:

- Métodos de interacción

1. Carga manual de parámetros individuales mediante formulario (GET request):

Se permite ingresar manualmente los valores de cada una de las variables predictoras a través de un formulario en Swagger UI. Cada campo corresponde a una característica de la canción (por ejemplo, **duration_ms**, **danceability**, **energy**, **tempo**, etc.).

Este método es práctico para realizar predicciones rápidas sobre un único registro.

• Ejemplo 1:

Se cargaron manualmente los datos correspondientes al **registro 1** del **set de validación** (`dataTesting`) de Spotify, mediante el formulario de Swagger. El `track_id` del registro es **6KwkVtXm8OUp2XffN5k7IY**.

La API devolvió una **popularidad estimada de 46** para esta canción.

The screenshot displays a REST client interface with two main panels. The left panel shows a GET request to `/predict/params` with various query parameters like `tempo`, `explicit`, `key`, `mode`, `time_signature`, `artists`, `album_name`, `track_name`, and `track_genre`. The right panel shows the response, which is a JSON object containing a single key: `"popularidad_estimada": 46`. The response status is 200, and the content type is `application/json`.

2. Envío de datos estructurados en formato JSON (POST request):

Se permite enviar el conjunto completo de características en formato JSON a través de una solicitud POST. Esta modalidad es adecuada para cargas automáticas, envío masivo de datos o integración con otros sistemas.

• Ejemplo 2:

Se utilizó el **registro 1.228** del **set de validación** (dataTesting) para construir un objeto JSON que fue enviado mediante POST a la API, el `track_id` del registro es **51TG9W3y9qyO8BY5RXKgnZ**.

La API devolvió una **popularidad estimada de 66** para esa canción.

The screenshot displays a REST client interface with two main panels. The left panel shows a POST request to `/predict/` with a JSON payload in the body. The payload is a JSON object containing various features like `tempo`, `explicit`, `key`, `mode`, `time_signature`, `artists`, `album_name`, `track_name`, and `track_genre`. The right panel shows the response, which is a JSON object containing a single key: `"popularidad_estimada": 66`. The response status is 200, and the content type is `application/json`.

Interpretación de los resultados

En ambos casos, los registros utilizados para la predicción fueron extraídos del **set de validación (dataTest_Spotify_csv)**, es decir, de datos no utilizados durante el entrenamiento del modelo. Esto garantiza que las predicciones reflejan la verdadera capacidad de generalización del modelo sobre observaciones nuevas y no vistas. La API procesó correctamente las solicitudes y devolvió las predicciones en formato JSON, cumpliendo con los requisitos de eficiencia, consistencia y disponibilidad requeridos para servicios de machine learning en producción.

5. Conclusiones

• Éxito en el procesamiento y limpieza de datos:

Se logró un preprocesamiento de datos riguroso que incluyó la verificación de tipos de variables, análisis de valores faltantes, inspección de correlaciones y correcta separación de conjuntos de entrenamiento y validación.

Estas acciones sentaron una base sólida para el desarrollo del modelo, minimizando riesgos de inconsistencias, ruido o sesgos en los datos. La alta calidad inicial del dataset permitió evitar procesos de imputación o transformaciones complejas, acelerando el flujo de trabajo y reduciendo la probabilidad de errores de modelado.

- **Desarrollo de un modelo robusto y calibrado adecuadamente:**

El modelo final, basado en **CatBoostRegressor**, alcanzó un desempeño competitivo tras la calibración exhaustiva de hiperparámetros clave como `iterations`, `depth`, `learning_rate` y `l2_leaf_reg`. Con un RMSE de aproximadamente **9.42** en el conjunto de validación y un R^2 cercano a **0.83**, se demuestra una buena capacidad de predicción y generalización, adecuada para las características de variabilidad propias de datos musicales. Además, el uso de técnicas como **early stopping** permitió prevenir el sobreajuste, optimizando el equilibrio entre sesgo y varianza.

- **Justificación sólida en la selección del modelo:**

La elección de **CatBoost** resultó estratégica, dado su manejo nativo de variables categóricas de alta cardinalidad sin necesidad de codificación manual, su resistencia a escalas distintas entre variables y su eficiencia computacional. Estas características hicieron que el modelo fuera particularmente adecuado para el problema tratado, donde campos como **artists**, **album_name** y **track_name** hubieran requerido preprocesamientos complejos en otros algoritmos.

- **Comparación y validación exhaustiva del modelo:**

El modelo fue comparado frente a alternativas como **XGBoost**, **LightGBM**, **Random Forest** y **Gradient Boosting**, demostrando consistentemente mejores métricas de desempeño. Esto valida empíricamente que la selección de CatBoost no fue arbitraria, sino fundamentada en pruebas sistemáticas de desempeño mediante validación cruzada K-Fold.

- **Disponibilización exitosa del modelo en entorno cloud:**

El despliegue de la API mediante Flask en una instancia EC2 de AWS demostró competencias no solo en machine learning, sino también en ingeniería de software y servicios en la nube. Se construyó una API funcional, documentada (Swagger UI) y persistente, capaz de atender solicitudes externas de predicción, evidenciando la capacidad de trasladar proyectos de laboratorio a entornos de producción real.

- **Validación de la API mediante pruebas funcionales:**

Se realizaron predicciones exitosas sobre nuevas observaciones mediante solicitudes POST y GET, confirmando que el modelo era capaz de recibir datos externos, procesarlos correctamente y retornar predicciones consistentes en un entorno de producción.

- **Importancia de la reproducibilidad y buenas prácticas:**

El uso sistemático de semillas (`random_state=42`), la separación controlada de datos, la documentación del modelo y el control de versiones garantizan que todo el flujo de trabajo pueda ser reproducido por otros desarrolladores o auditores. Esto refuerza la transparencia y la confiabilidad del proceso.

- **Retos identificados y oportunidades de mejora:**

Durante el desarrollo del proyecto se identificaron posibles líneas de mejora, como la exploración de técnicas avanzadas de regularización o la incorporación de mayor volumen de datos para robustecer el entrenamiento. A futuro, se podría evaluar también la implementación de pipelines automáticos de actualización de modelos en la nube.

- **Cumplimiento exitoso de los objetivos del proyecto:**

El modelo predictivo fue construido, calibrado, evaluado y desplegado exitosamente, cumpliendo cada una de las etapas requeridas: desde la preparación de datos hasta la disponibilización de un servicio de predicción funcional y documentado, lo cual demuestra la consecución efectiva de los objetivos planteados al inicio del proyecto.